

Smart Asset Management and Resource Allocation Platform by : Devansh Saxena 24115052

Design Document

1. Problem Understanding

Organizations heavily rely on shared resources, but managing them through fragmented channels (like WhatsApp, spreadsheets, and physical registers) leads to operational bottlenecks. The core problem is a lack of **real-time visibility and state synchronization**. When physical assets (like DSLR cameras or studio lights) are moved without immediate digital tracking, it results in double-booking, ghost inventory, and zero accountability for overdue items. This platform solves that by enforcing a strict, role-based digital handshake between the resource consumer (Student) and the resource manager (Administrator), ensuring the digital state always reflects physical reality.

2. System Architecture

To optimize for rapid prototyping and data integrity, the system is built using a **Monolithic Stateful Architecture**.

- **Frontend & Application Layer:** Built with Streamlit (Python). This allows for rapid, reactive UI components that communicate directly with the data layer without the overhead of standard HTTP request/response lifecycles.
- **Data Layer:** SQLite3. A lightweight, ACID-compliant relational database.
- **Authentication:** Stateful session management utilizing Python's hashlib for SHA-256 password encryption.

3. Database Schema

The relational database operates on three core tables to ensure data normalization and prevent redundant records.

Table 1: users

- username (TEXT) - Primary Key
- password (TEXT) - SHA-256 Hashed
- role (TEXT) - Enum: 'Administrator', 'Student / User'

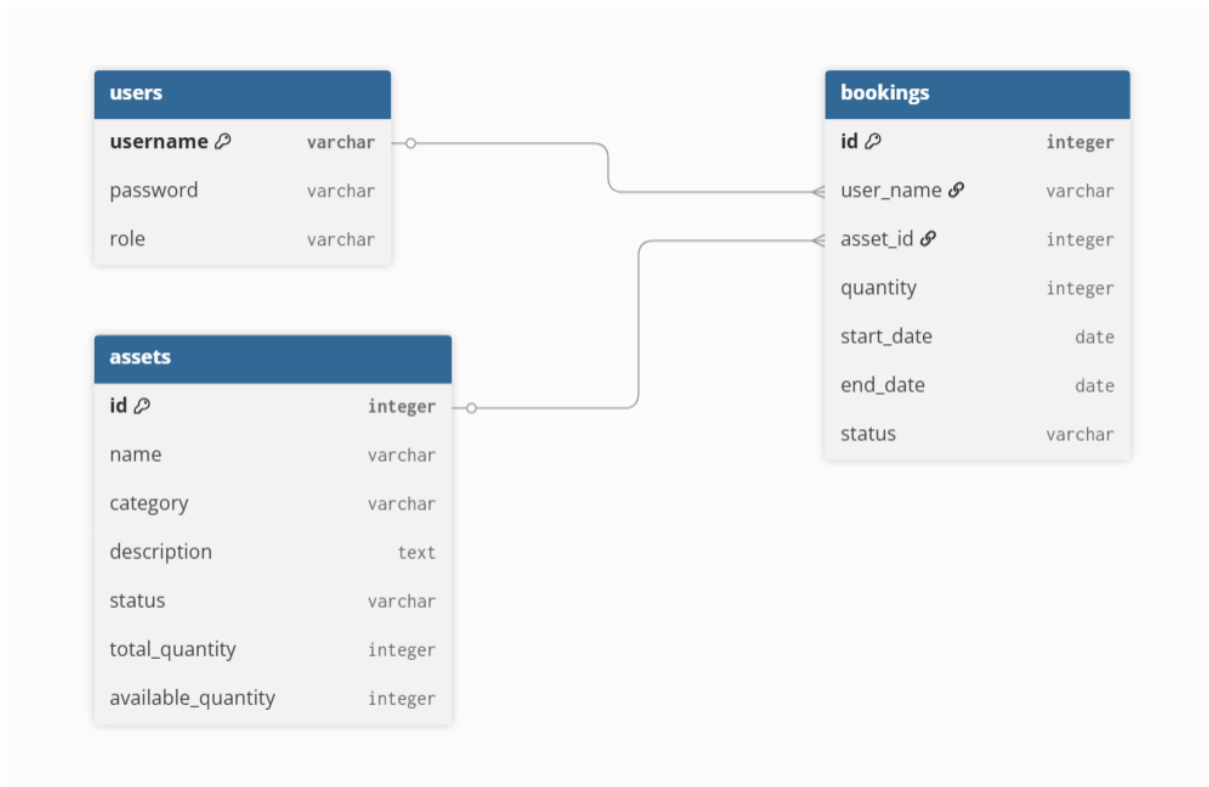
Table 2: assets

- id (INTEGER) - Primary Key, Auto-increment
- name (TEXT) - Name of the item
- category (TEXT) - Grouping classification
- description (TEXT) – Specifications
- status (TEXT) - Enum: 'Active', 'Under Maintenance', 'Retired'
- total_quantity (INTEGER) - Total physical count owned
- available_quantity (INTEGER) - Live count currently in storage

Table 3: bookings

- id (INTEGER) - Primary Key, Auto-increment
- user_name (TEXT) - Foreign Key referencing users(username)
- asset_id (INTEGER) - Foreign Key referencing assets(id)
- quantity (INTEGER) - Amount requested
- start_date (TEXT) - Date of checkout
- end_date (TEXT) - Expected return date
- status (TEXT) - Enum: 'pending', 'approved', 'rejected', 'returned'

4. Entity Relationship Diagram (ERD)



5. API Overview

Because the application utilizes a tightly coupled Streamlit architecture, it does not expose traditional REST endpoints (GET/POST/PUT/DELETE) over HTTP. Instead, it utilizes an Internal Python Database API via parameterized SQL queries.

- State Retrieval: Data is fetched using `pandas.read_sql_query()`, dynamically converting SQL relational tables into in-memory dataframes for charting and dashboard rendering.
- State Mutation: Transactions (Approvals>Returns) use `strict sqlite3.Cursor.execute()` statements with tuple bindings `(?, ?)` to prevent SQL Injection attacks. Dual-table updates (e.g., approving a booking and simultaneously reducing available inventory) are wrapped in single `commit()` blocks to maintain transactional integrity.

6. Design Decisions

1. **Framework Choice (Streamlit over React/Node):** Given the time constraint, traditional web stacks introduce heavy boilerplate. Streamlit allows pure Python logic to handle both

frontend state and backend processing, eliminating the need to write JSON APIs and cutting development time by 70%.

2. **Two-Step Physical Verification:** We deliberately did not allow students to click "Return" on their own dashboards. In inventory logistics, digital state must match physical state. Only an Admin can mark an item as "Returned," enforcing a physical handover of the equipment.
3. **Dynamic UI Rendering:** The platform dynamically hides invalid actions. For example, the dropdown menu for asset requests dynamically filters out any item where `available_quantity == 0`, proactively preventing out-of-stock errors before a user even submits a form.
4. **Immutable Security Audit Logging (Bonus Feature):** To address enterprise compliance requirements, a dedicated `audit_logs` table was implemented. A centralized helper function silently records all critical state mutations (e.g., `CREATE_ASSET`, `APPROVE_BOOKING`, `RETURN_ASSET`) alongside timestamps and the executing user's session ID. This prevents unauthorized inventory manipulation and creates a transparent paper trail for the Cultural Council administrators.